

Introduction to Deep Learning with PyTorch

Pytorch로 배우는 딥러닝 기초

딥러닝 모델 학습 원리

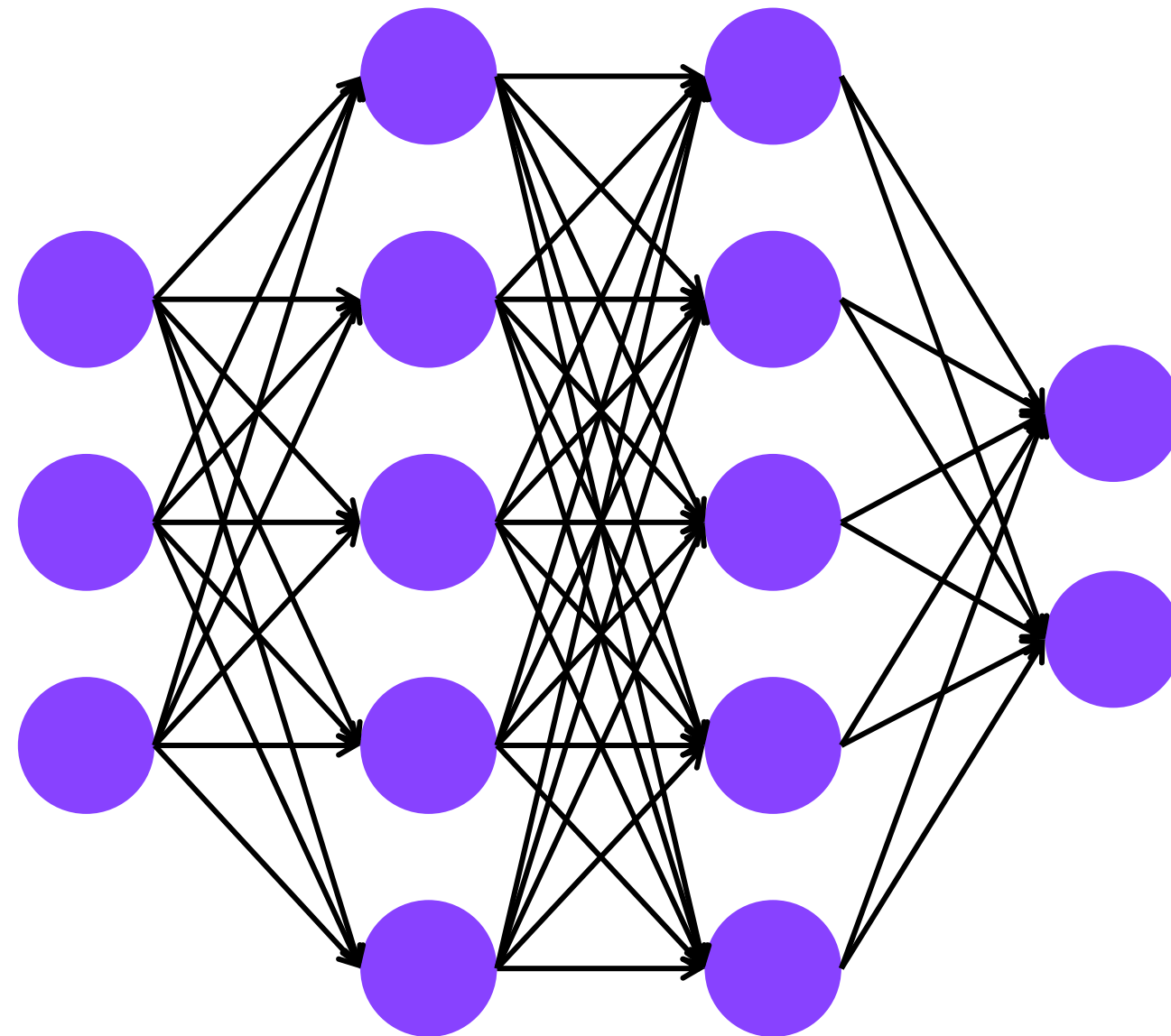
활성화 함수

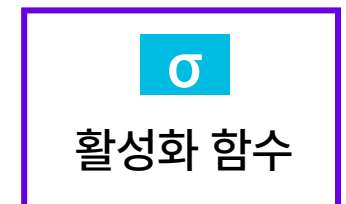
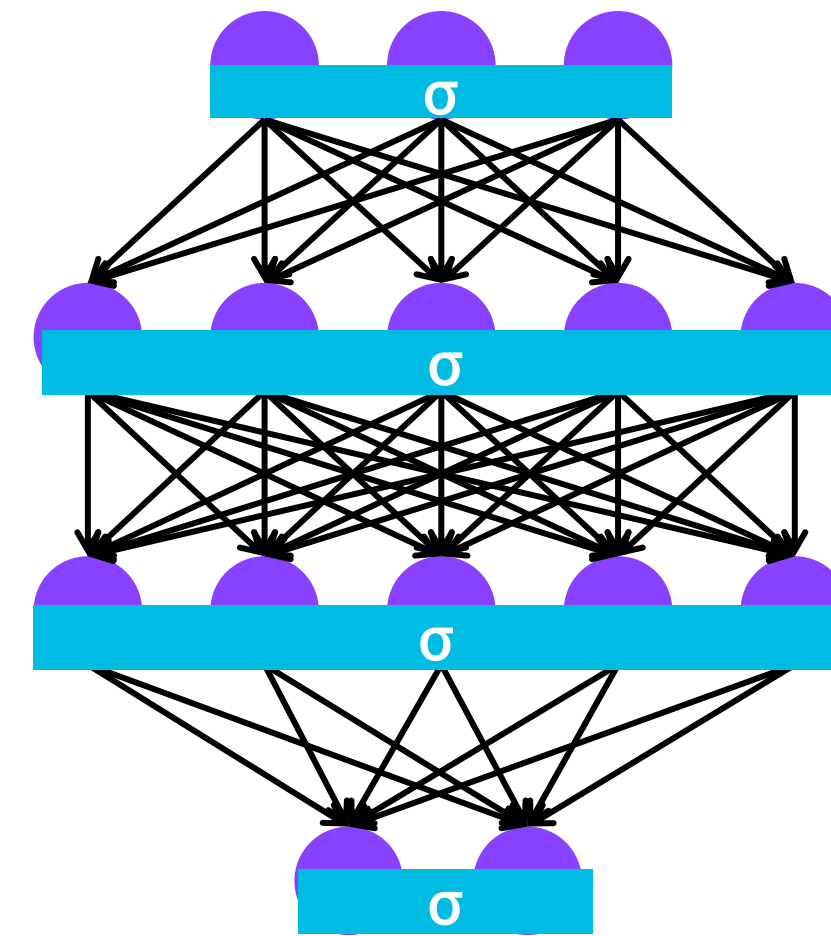
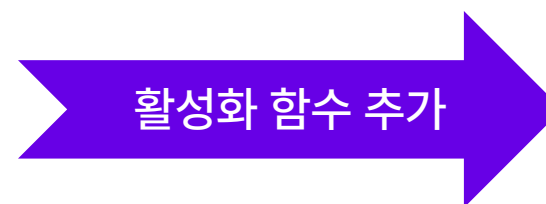
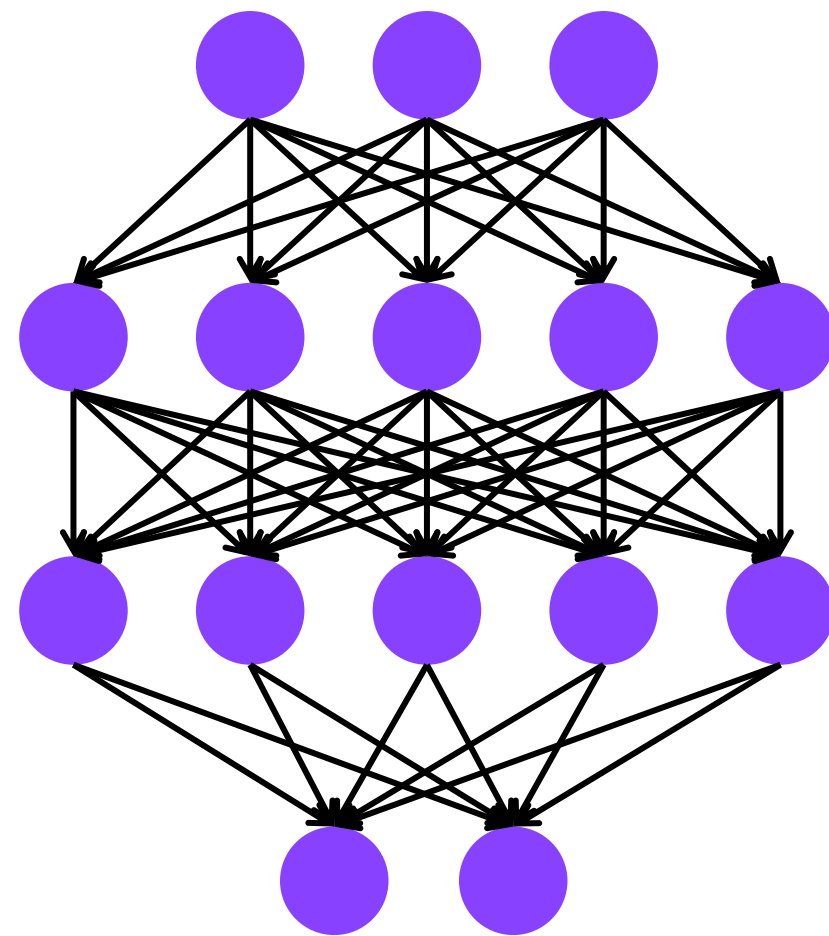


딥러닝의 강점

딥러닝의 **복잡한 함수를 잘 표현할 수 있는 성질**을 활용해 머신러닝의 한계를 넘을 수 있다.

> **활성화 함수를 포함**하여 복잡한 함수를 표현하기 위한 다양한 기법을 활용한다.

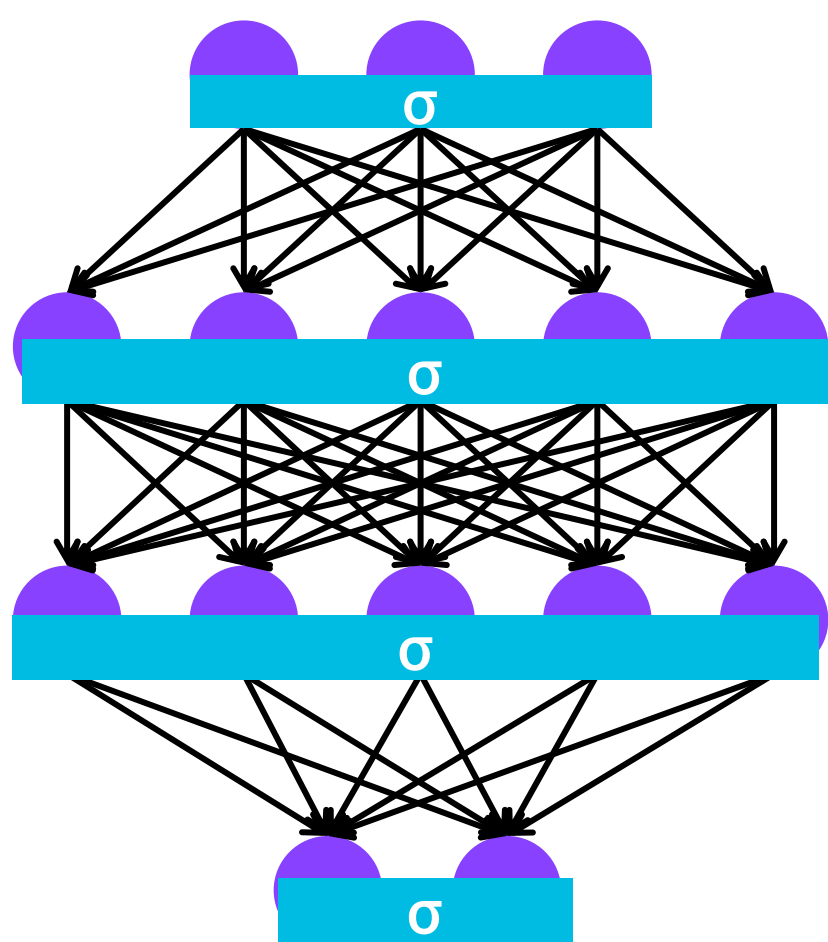




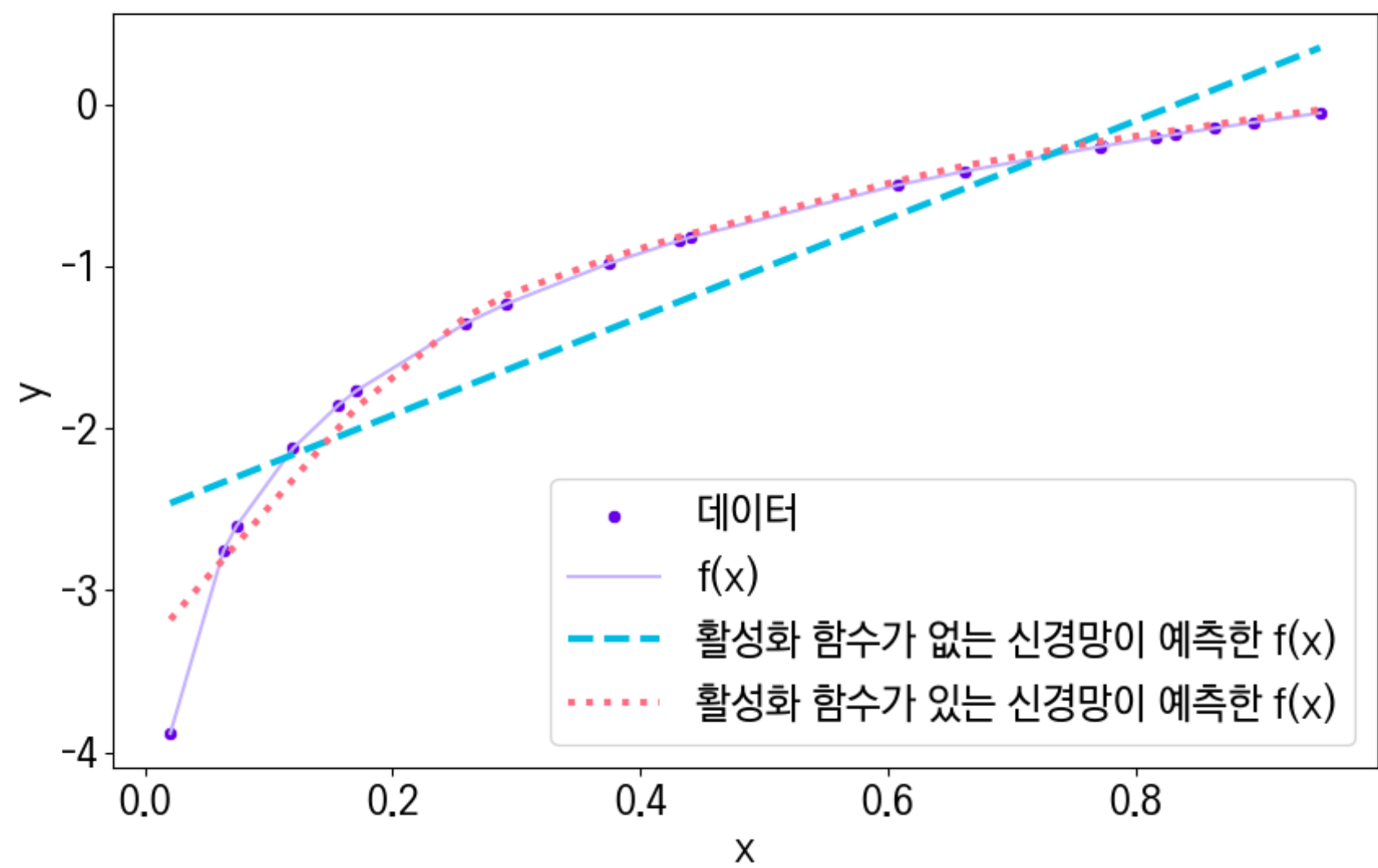
레이어의 연산이 끝나고 나가기 전에 거치는 함수
인공신경망이 **비선형성 관계**를 학습하게 하는 역할

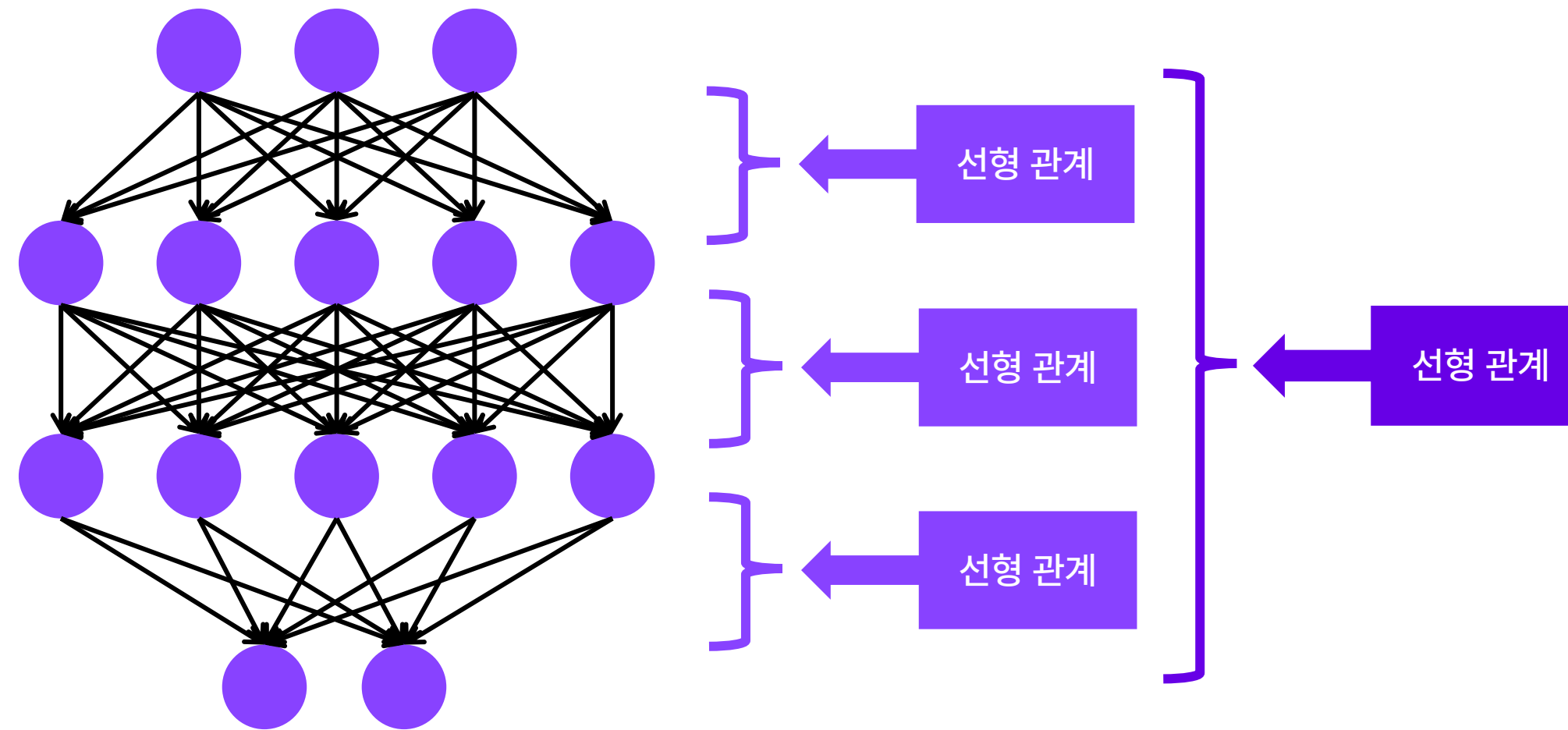


활성화 함수 - 비선형성



σ
활성화 함수





비선형성이 없는 선형 레이어의 중첩은 단일 선형 관계와 같음

비선형 회귀 등 **비선형성이 필요한** 문제를 풀 수 없음.



비선형성이 필요한 문제란?

문제 유형	예시	비선형성이 필요한 이유
이미지 분류	 → 고양이 / 개	귀, 눈, 꼬리 등 다양한 위치의 패턴이 조합되어 특징을 이룸 선형 모델은 픽셀 값의 단순 가중합으로는 복합적인 시각적 패턴을 구분할 수 없음
감성 분석	"not bad" → 긍정	문맥에 따라 단어의 의미가 바뀜 (예: "not"이 "bad"의 의미를 반전) 비선형성 없이는 이런 복잡한 상호작용을 표현할 수 없음
음성 인식	 → "안녕"	사람의 음성은 다양한 주파수, 억양, 속도 등 비선형적 특성을 가짐 소리의 스펙트럼 패턴과 의미 사이의 관계는 매우 복잡하고 비선형적

활성화 함수의 종류

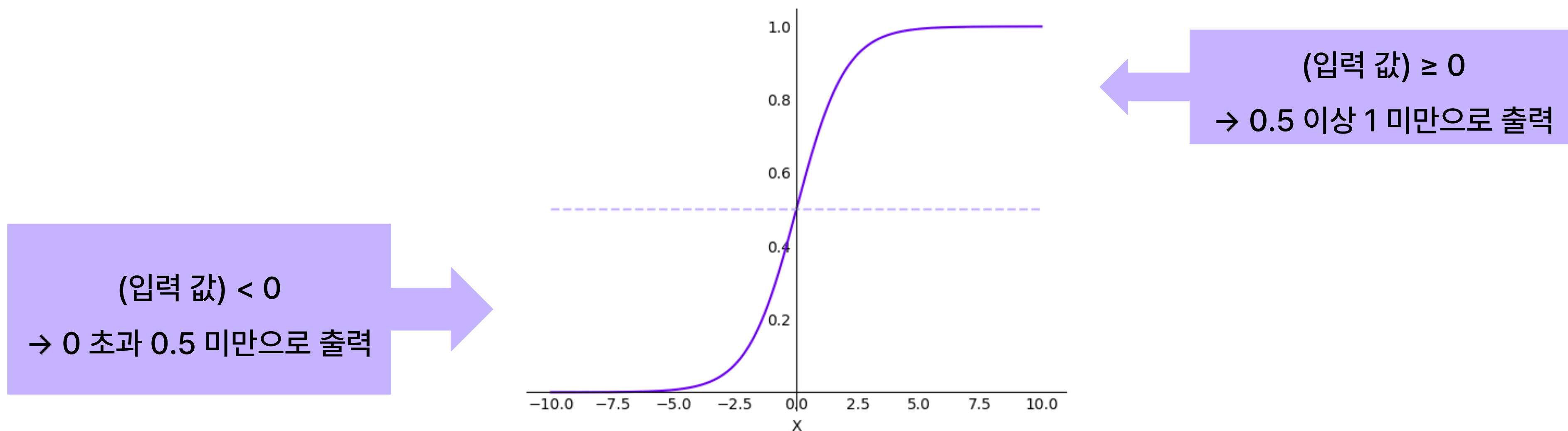


활성화 함수의 종류

일반적인 활성화 함수	분류 모델의 출력층
ReLU, LeakyReLU, Tanh ...	Sigmoid, Softmax ...

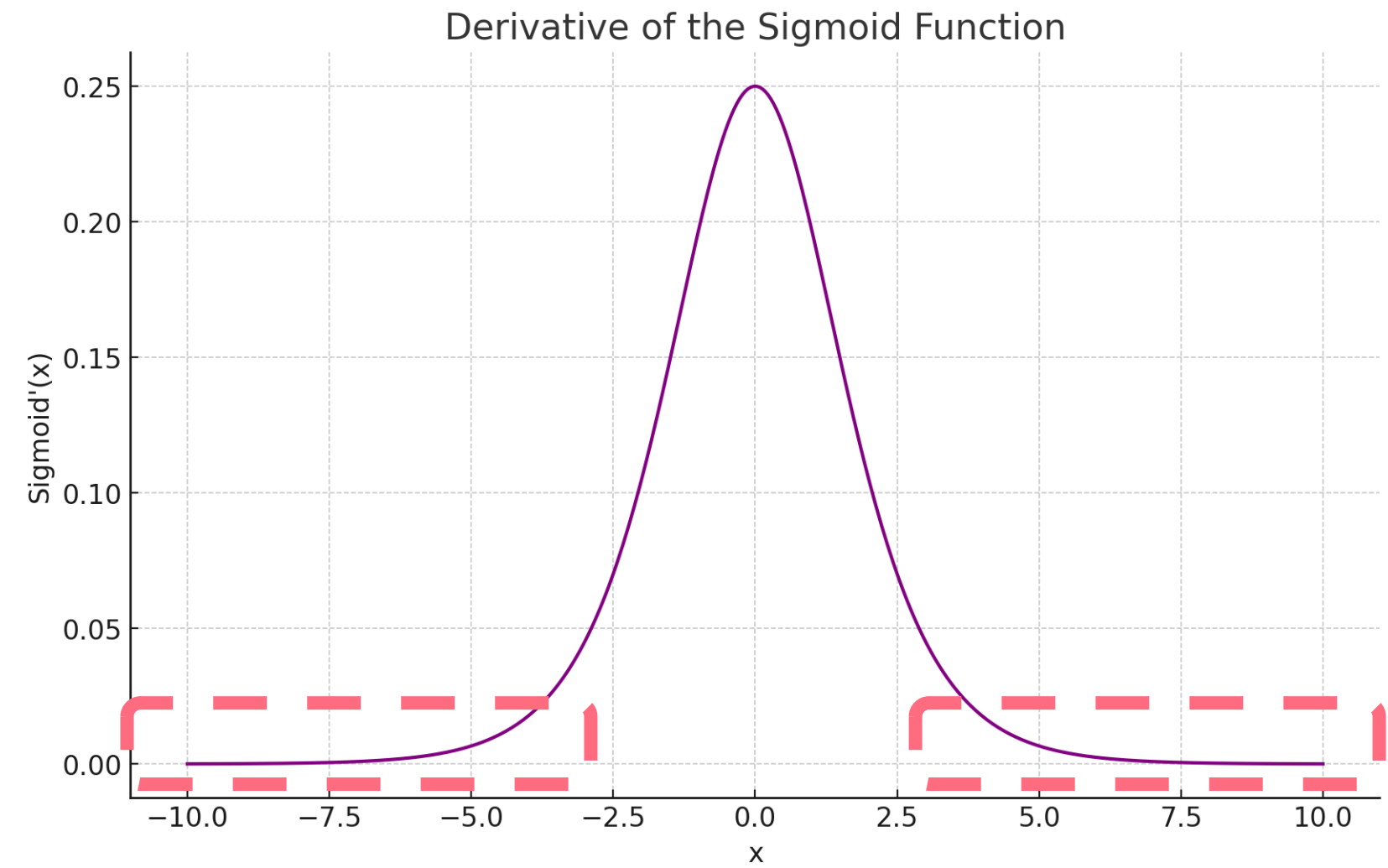
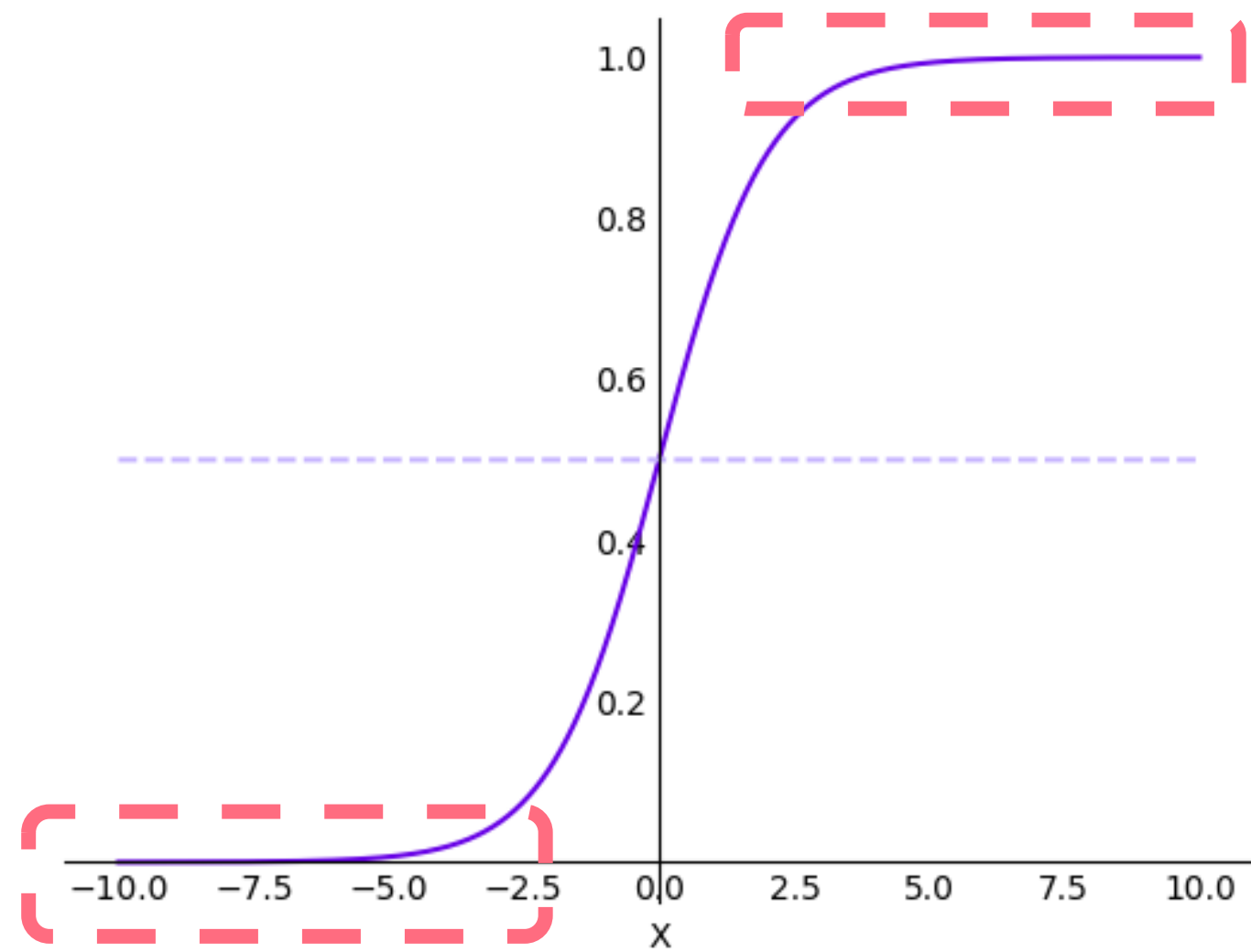


활성화 함수 - Sigmoid



입력 값을 0과 1 사이의 확률 값으로 변환하여 출력

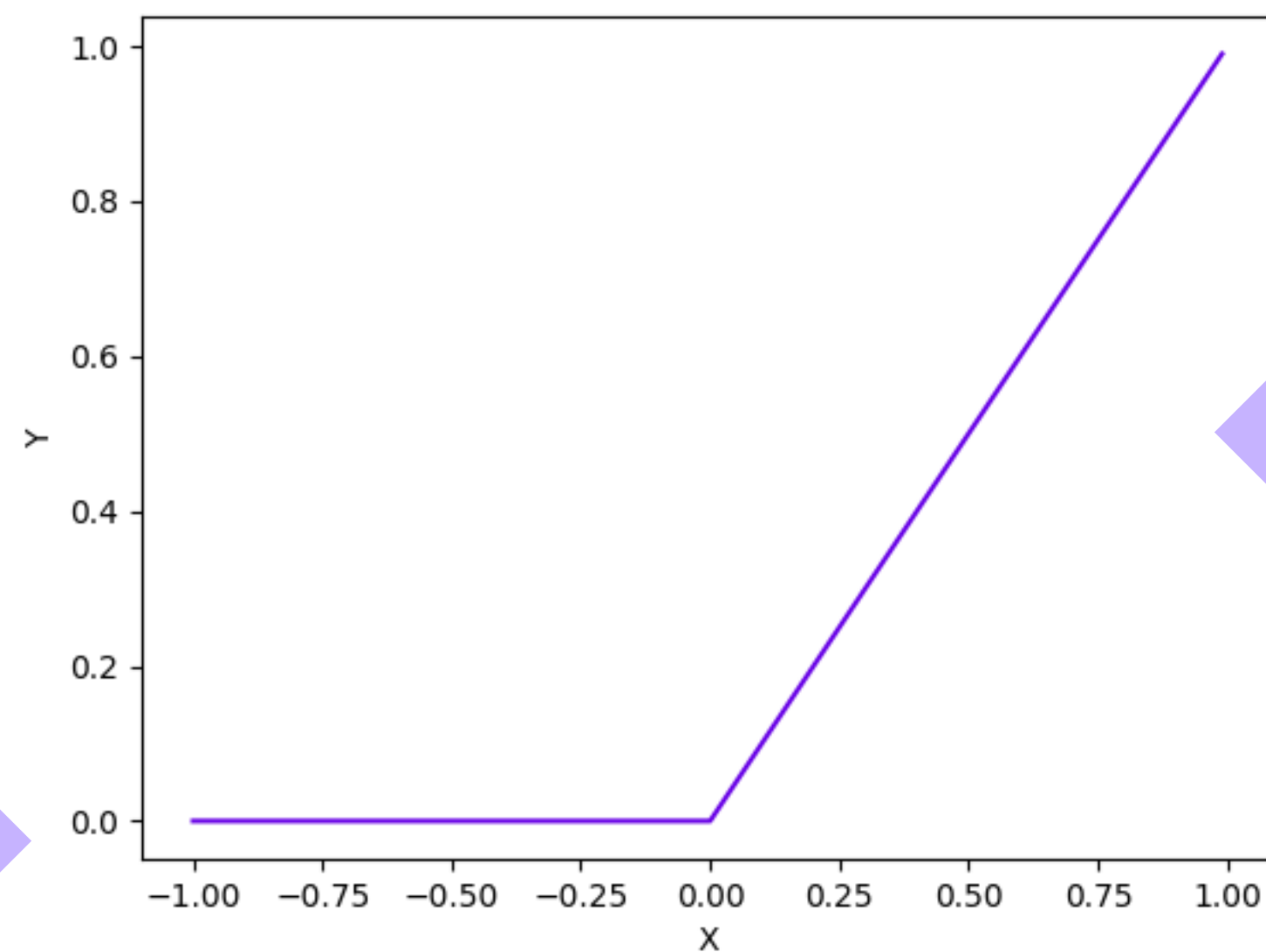
활성화 함수 – Sigmoid의 문제점



입력값의 절대값이 클 때 출력값의 변화가 매우 작다



활성화 함수 - ReLU



(입력 값) < 0
→ 0을 출력

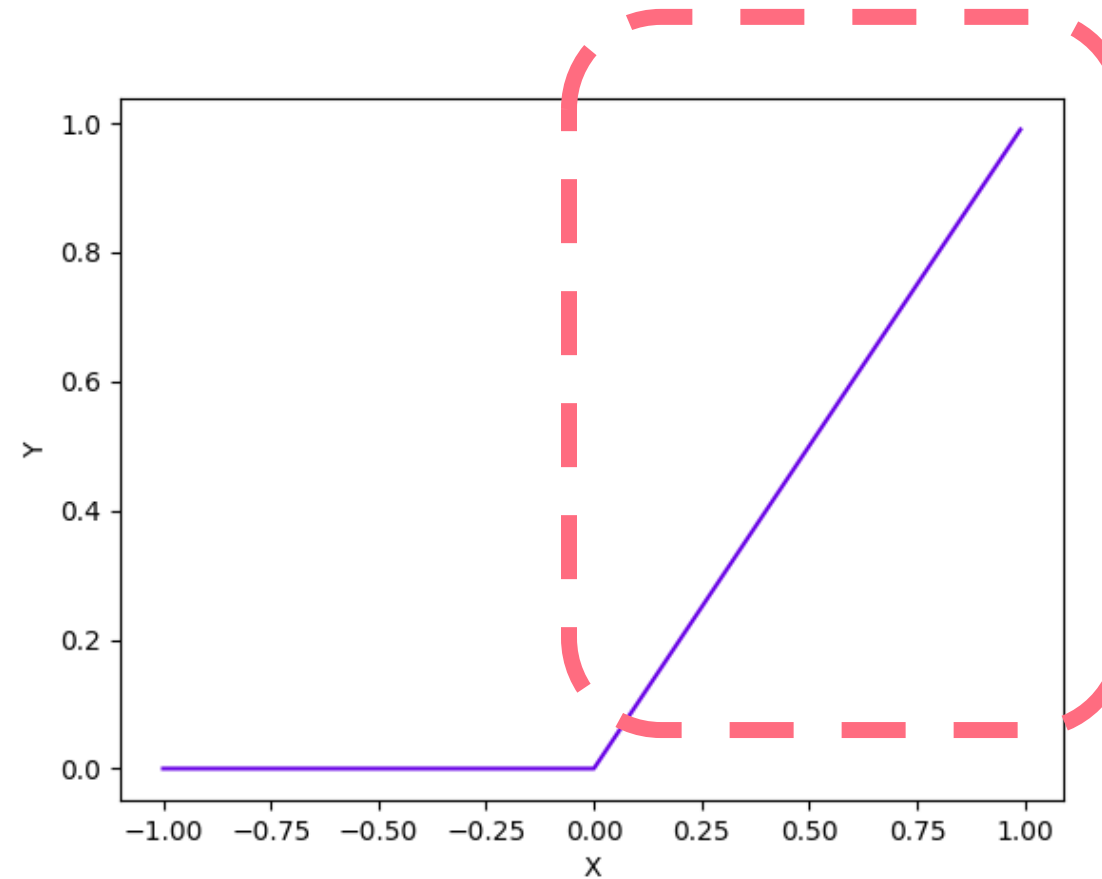
(입력 값) ≥ 0
→ 그대로 출력

은닉층에서 사용



활성화 함수 - ReLU

(입력 값) < 0
→ 0을 출력

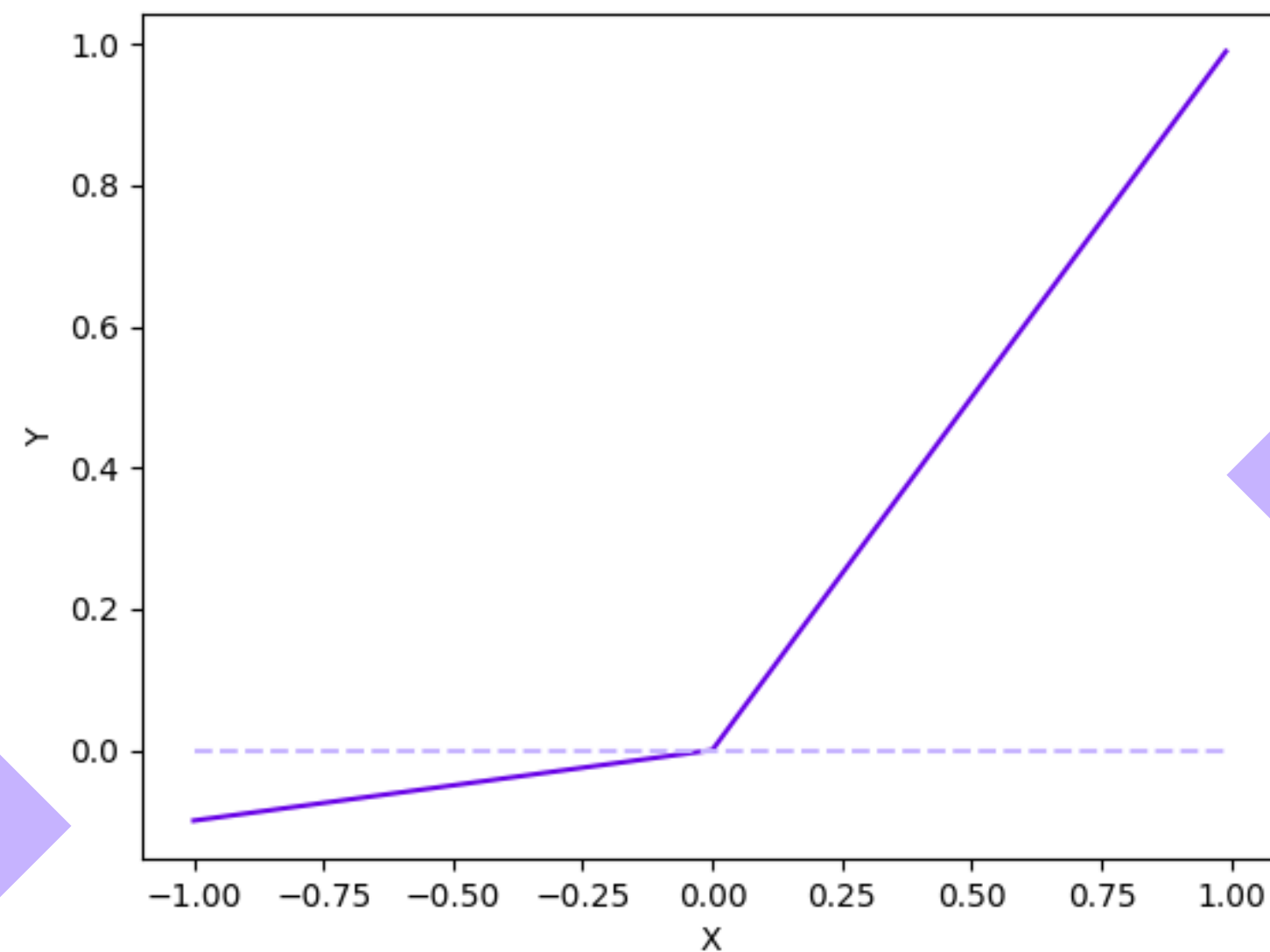


(입력 값) ≥ 0
→ 그대로 출력

입력값이 크더라도 출력값의 변화량이 줄어들지 않는다.
간단하면서도 효과적으로 **비선형성**을 부여한다.



활성화 함수 - LeakyReLU



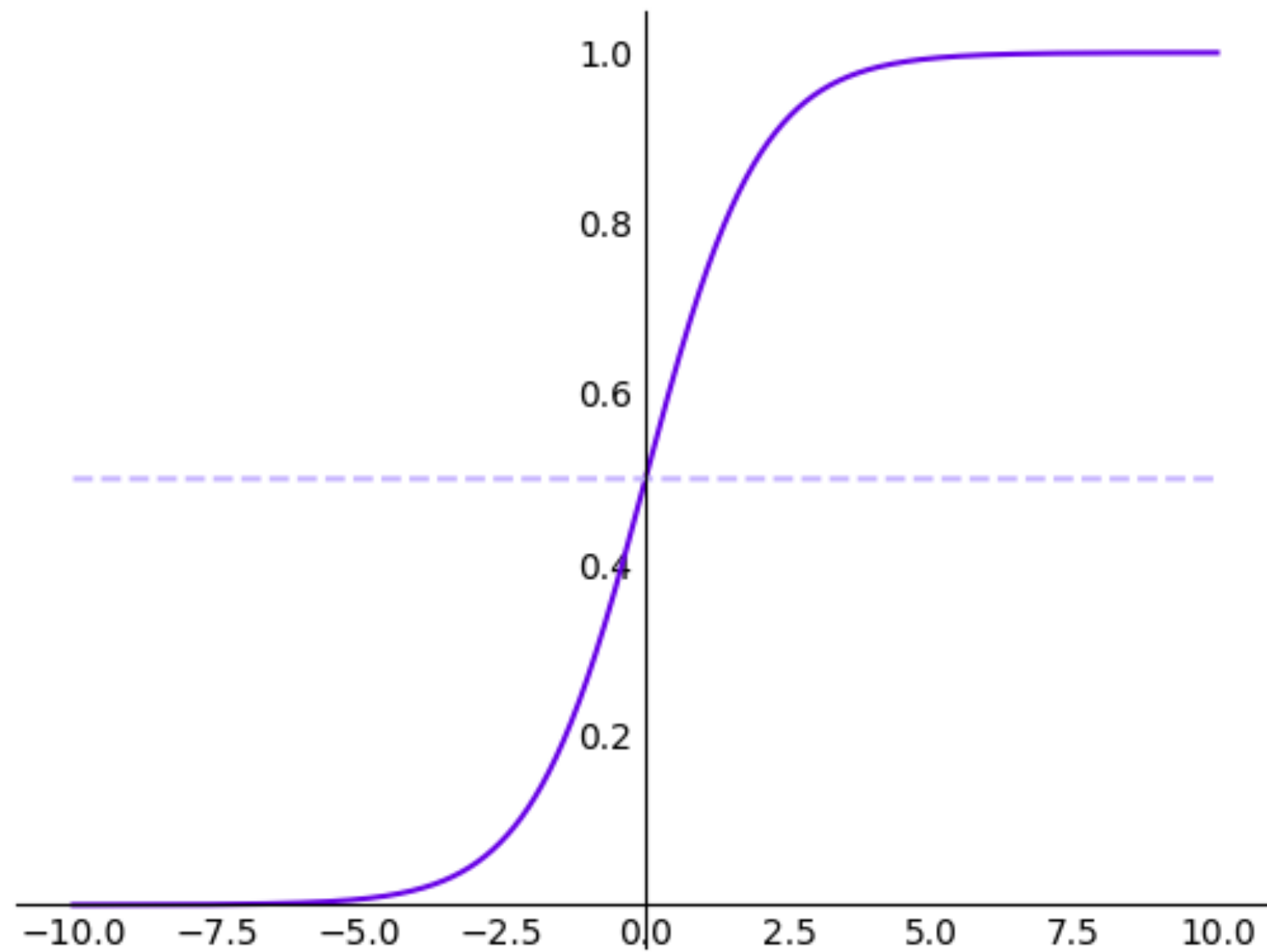
(입력 값) < 0
→ 입력 값의 일정 비율
(보통 0.01) 만 출력

(입력 값) ≥ 0
→ 그대로 출력

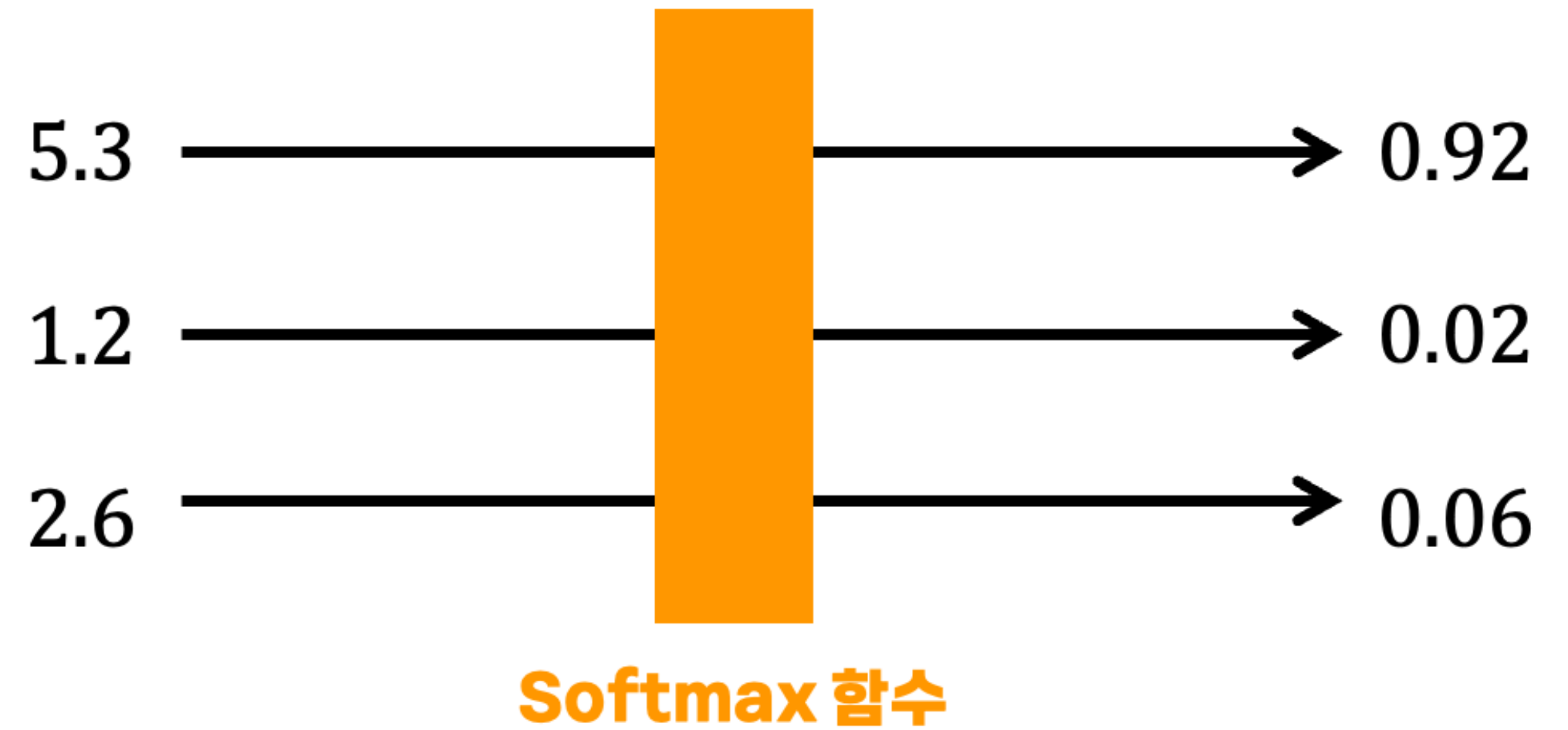
은닉층에서 사용



분류 모델 출력층의 활성화 함수



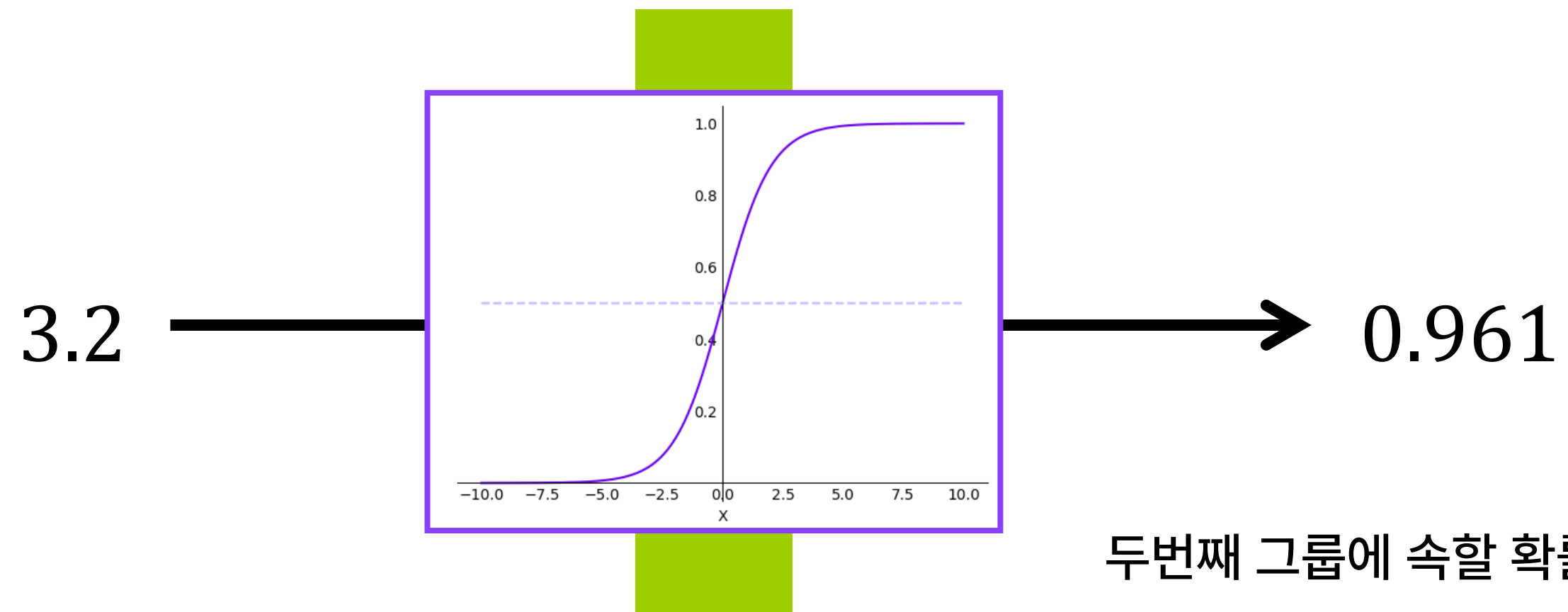
이진 분류 – Sigmoid 함수



다중 분류 – Softmax 함수



Sigmoid 함수의 출력 값의 의미

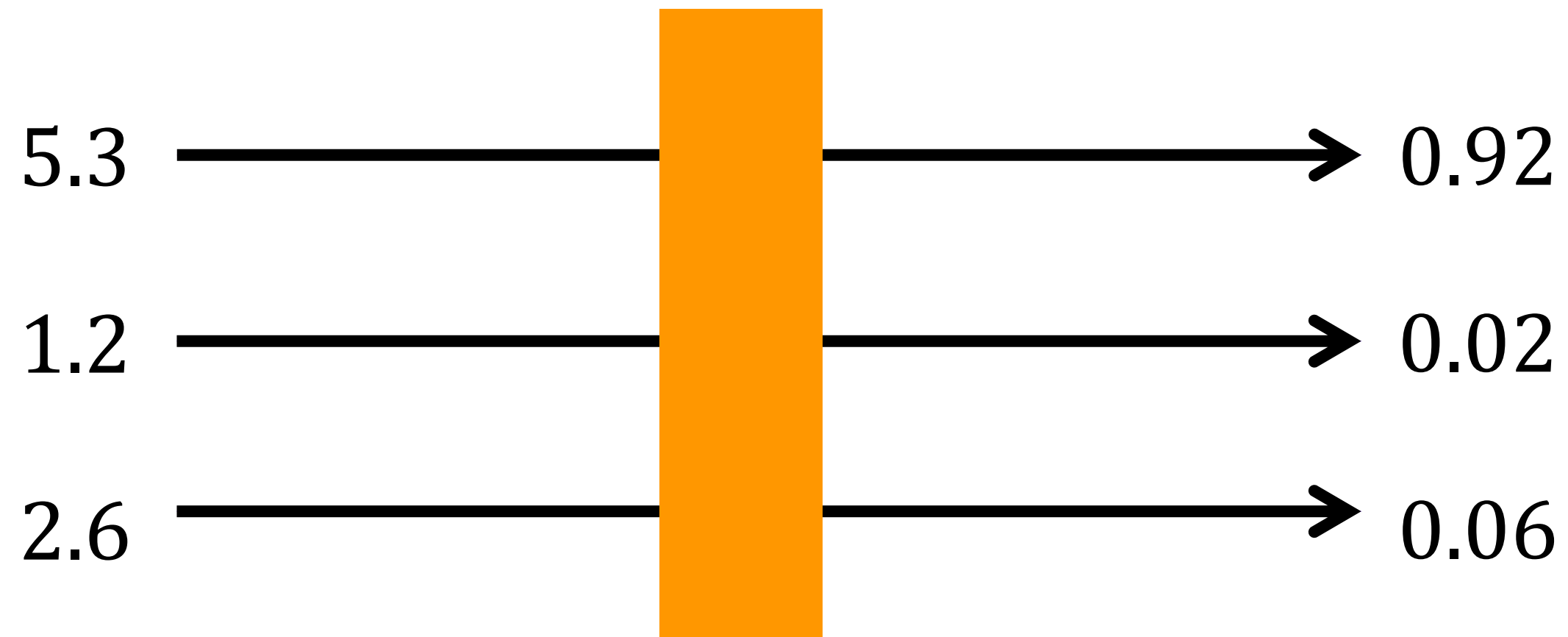


Sigmoid 함수

두번째 그룹에 속할 확률이 **96.1%**



Softmax 함수



Softmax 함수

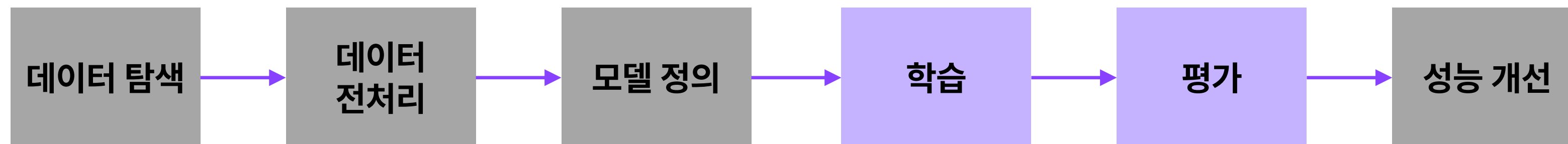
3개 이상의 입력 값을 각 분류에 속할 확률값으로 변환 후 출력

다중 분류 모델의 출력층에서 사용

손실 함수와 역전파



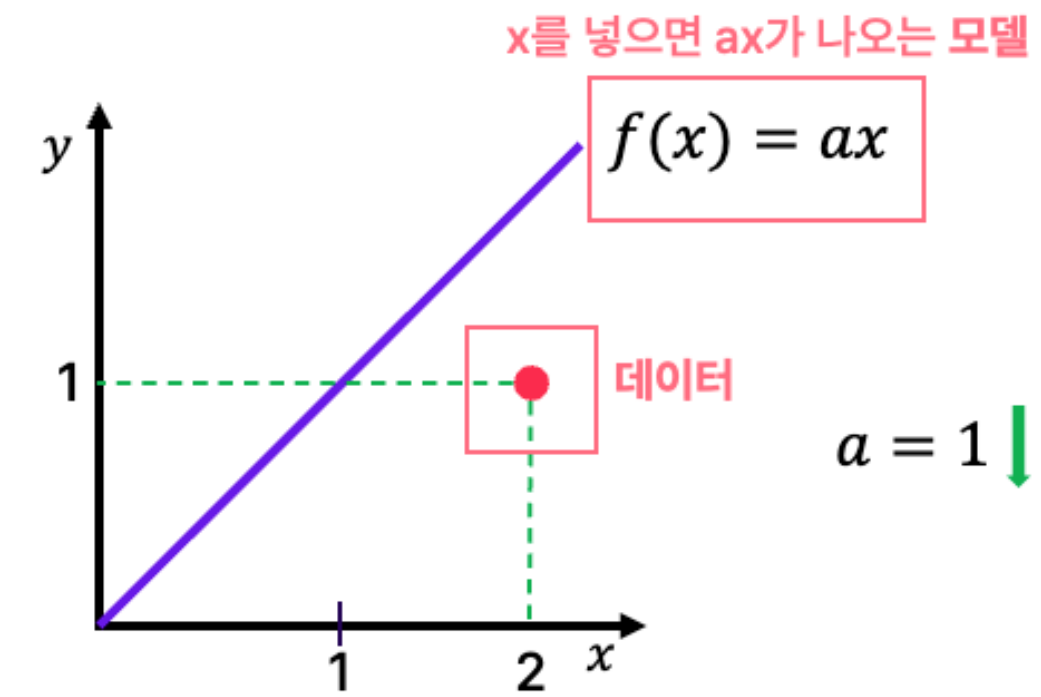
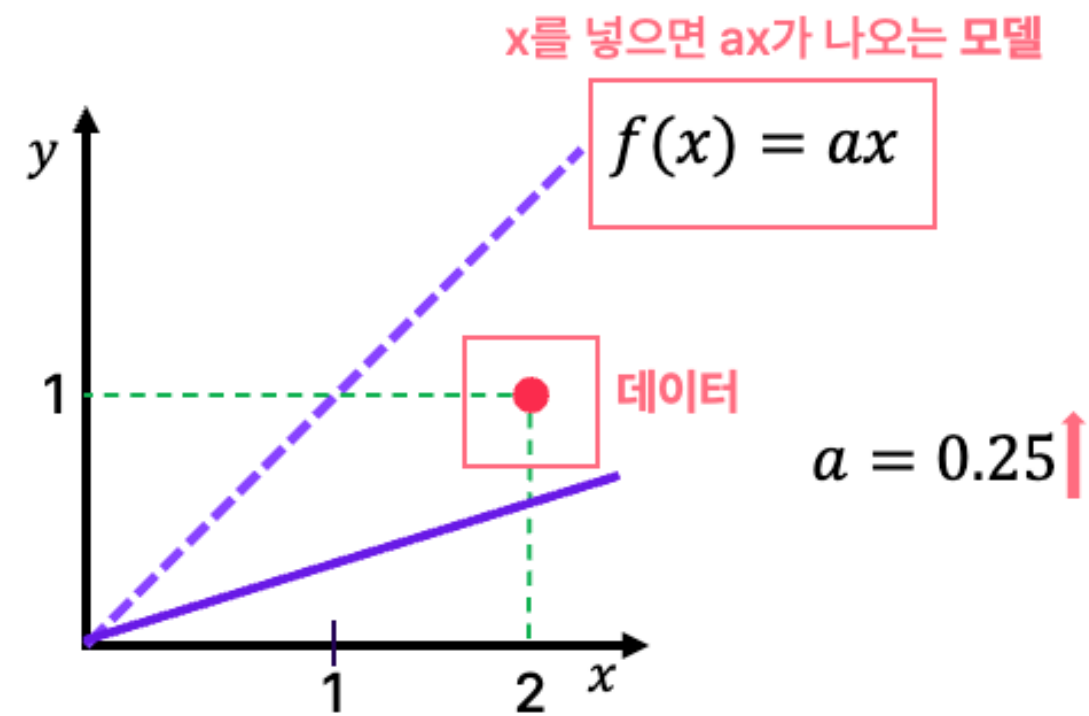
딥러닝 프로젝트 진행 과정



$$f(\text{고양이 이미지}) = \text{고양이}$$



딥러닝 모델의 학습



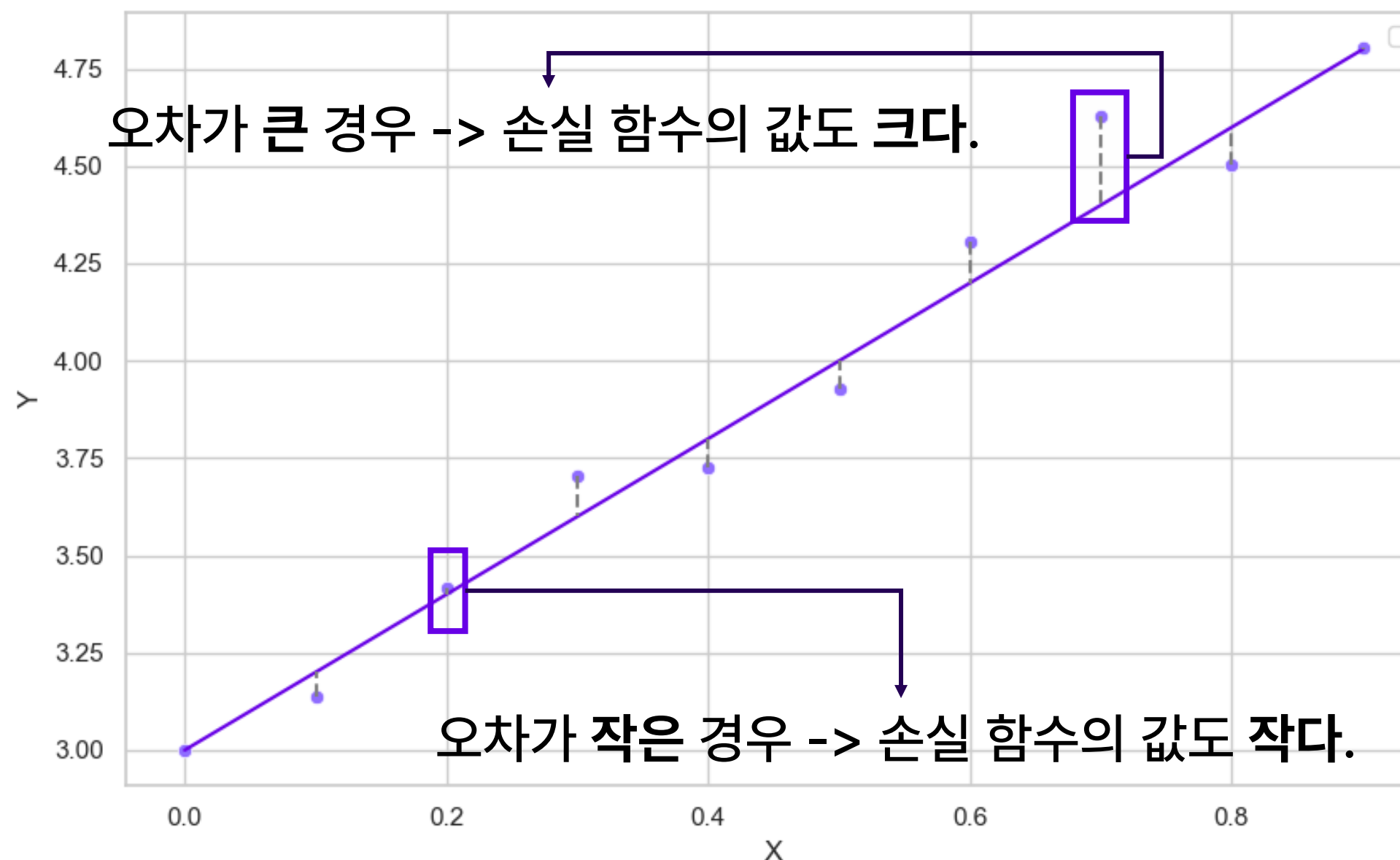
학습 과정에서 모델의 가중치를 조정함으로써,
사람이 수동으로 **특징 추출**을 하지 않고도 모델이 중요한 특징을 학습하게 할 수 있다.

머신러닝과 딥러닝 모델에서 예측값과 실제값 사이의 차이를 수치적으로 나타내는 함수

	회귀 모델	이진 분류 모델	다중 분류 모델
평가 방법	예측 값과 실제 값 간의 오차	분류 정확도, F1 Score 등	분류 정확도, F1 Score 등
손실 함수	MSE, MAE 등 오차 관련 함수	Binary Cross Entropy	Categorical Cross Entropy



손실함수 - 오차 관련 함수



MSE (Mean Squared Error)

오차의 제곱의 평균

MAE (Mean Absolute Error)

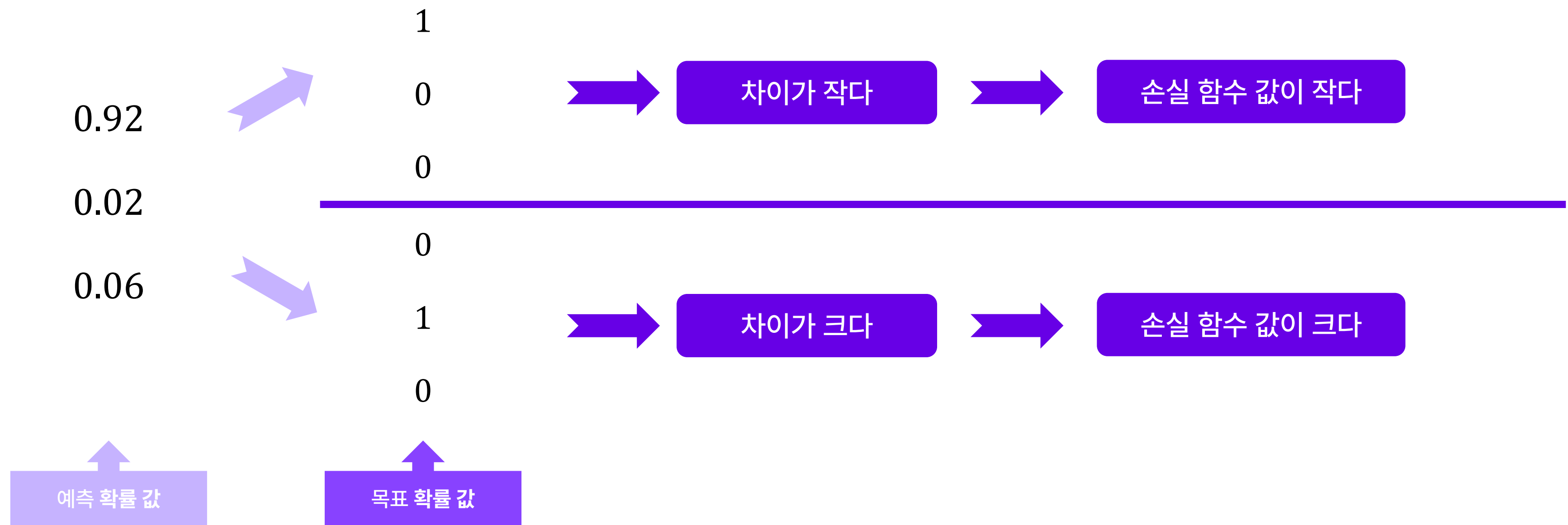
오차의 절댓값의 평균

RMSE (Root Mean Square Error)

오차의 제곱의 평균의 제곱근



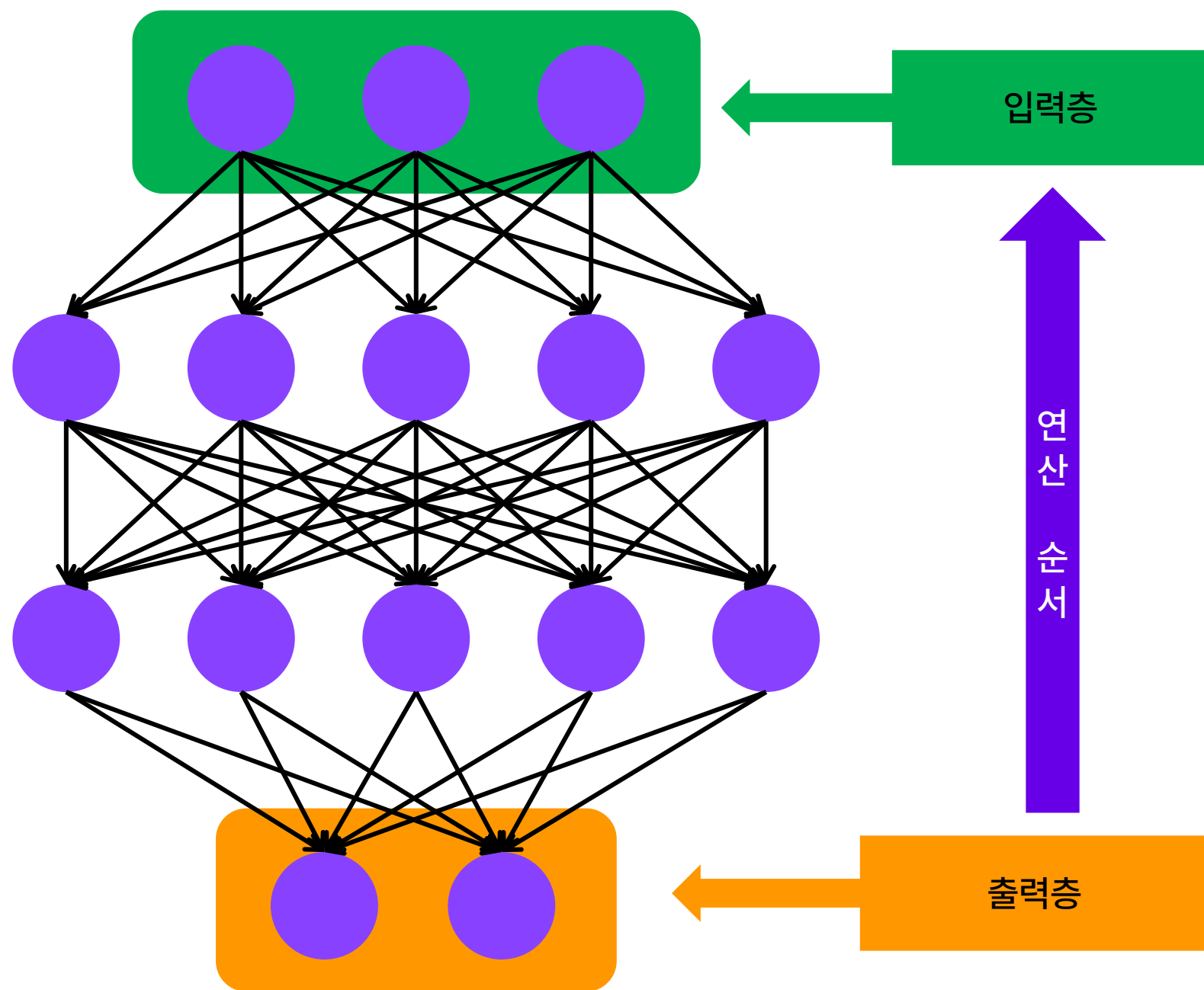
손실함수 – Cross Entropy



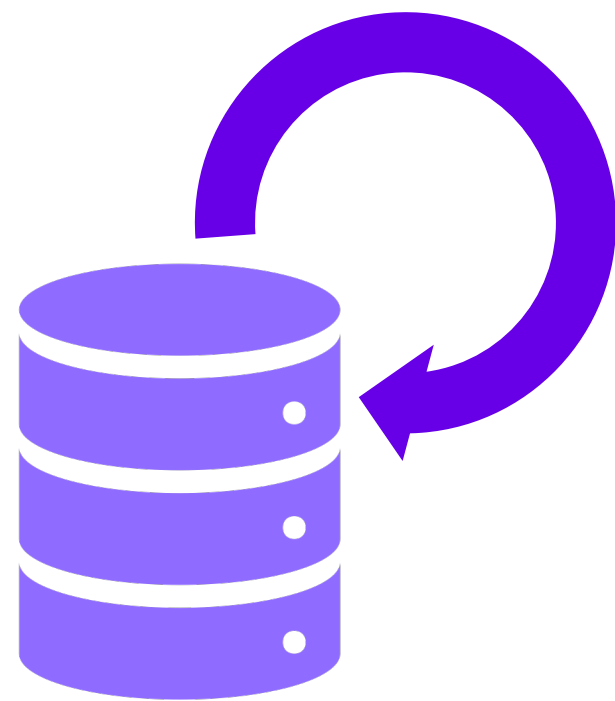
두 확률 값 사이의 차이를 나타내는 함수



역전파 (BackPropagation)



- 출력값과 실제 목표값 간의 오차를 최소화하는 방향으로, 출력층에서 입력층까지 가중치를 조정하는 과정
- 순전파 (Forward Propagation) : 입력층에서부터 출력층까지 데이터가 전달되며 추론, 예측을 진행하는 과정
- 가중치 조정을 통해 데이터를 학습



학습 데이터

모델이 전체 학습 데이터를 한 번 모두 사용해 학습하는 과정을 의미

- 학습 데이터가 1000장 이미지라고 가정
- 이 1000장을 한 번 다 보고 모델을 학습시키면 1 epoch
- 이걸 5번 반복하면 5 epochs

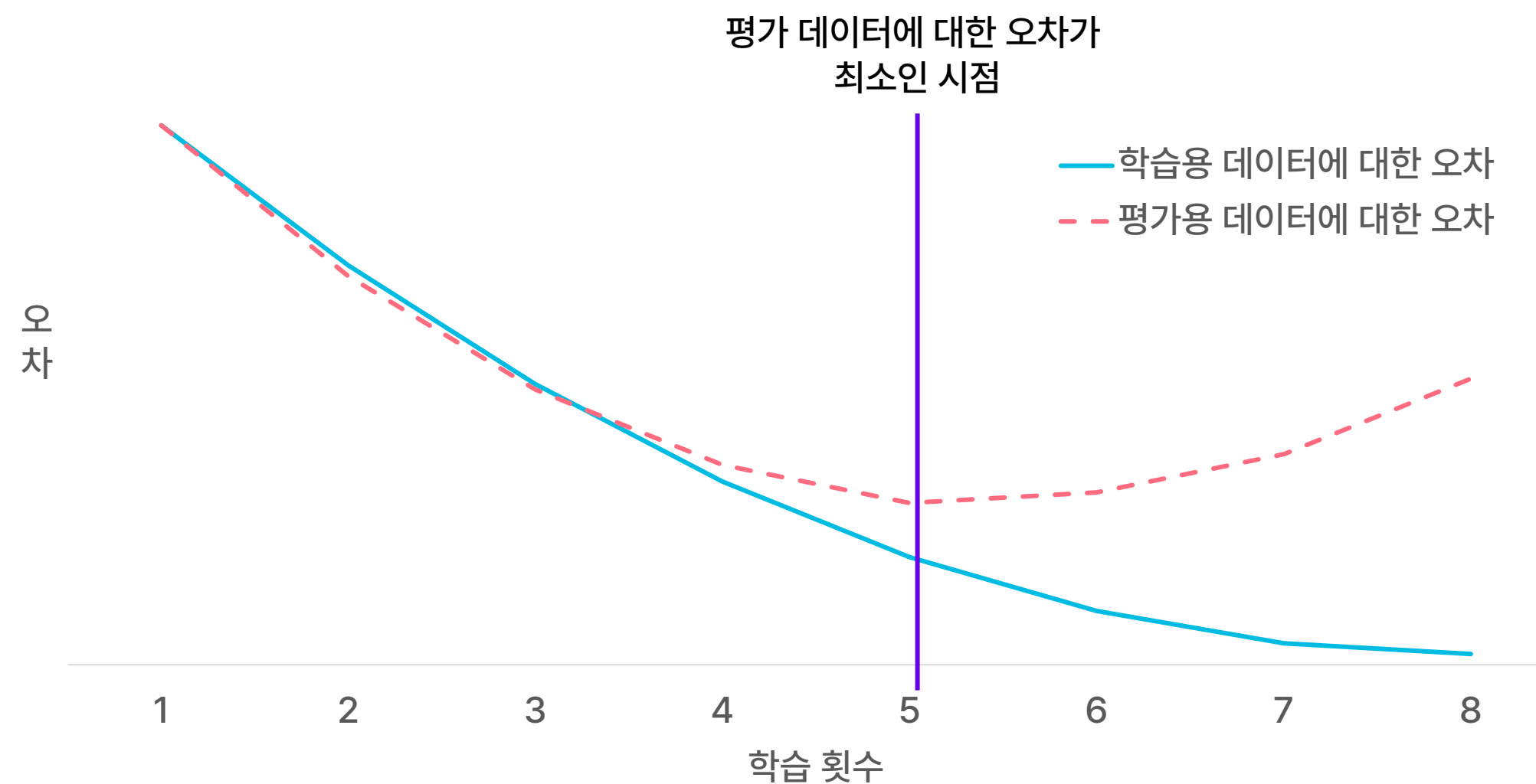
과적합과 정규화



모델이 학습하지 않은 새로운 데이터를 처리하는 성능



과적합 (Overfitting)



모델이 주어진 훈련 데이터에 과도하게 맞춰져
새로운 데이터가 입력 되었을 때 잘 예측하지 못하는 현상



검증 (Validation) 데이터



과정에서 검증을 위해 학습 데이터의 일부를 검증 데이터로 분리하여 사용
일반적으로 학습 데이터의 10~20%를 사용



딥러닝 모델 관점에서의 과적합 방지 기법

가중치 정규화

Regularization

조기 종료

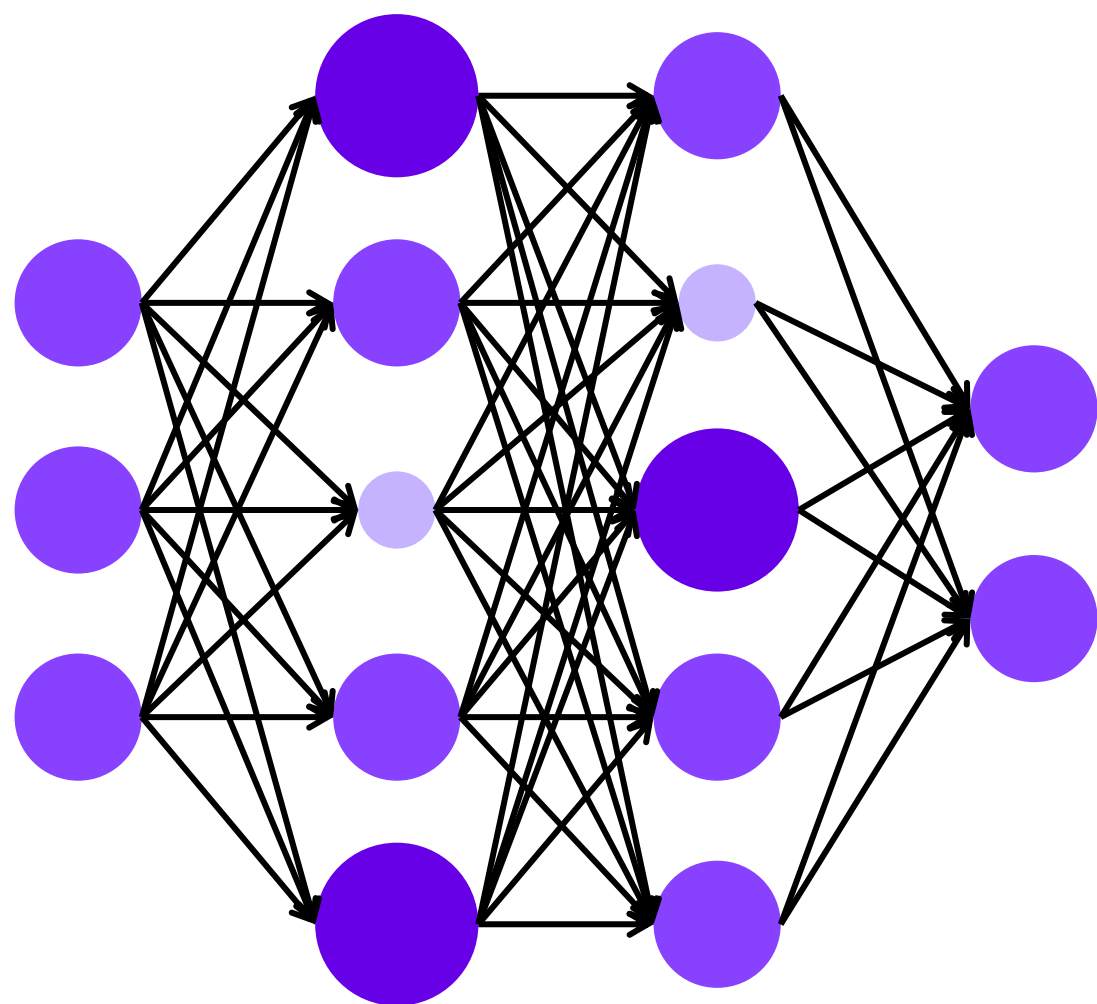
Early Stopping

드롭아웃

Dropout

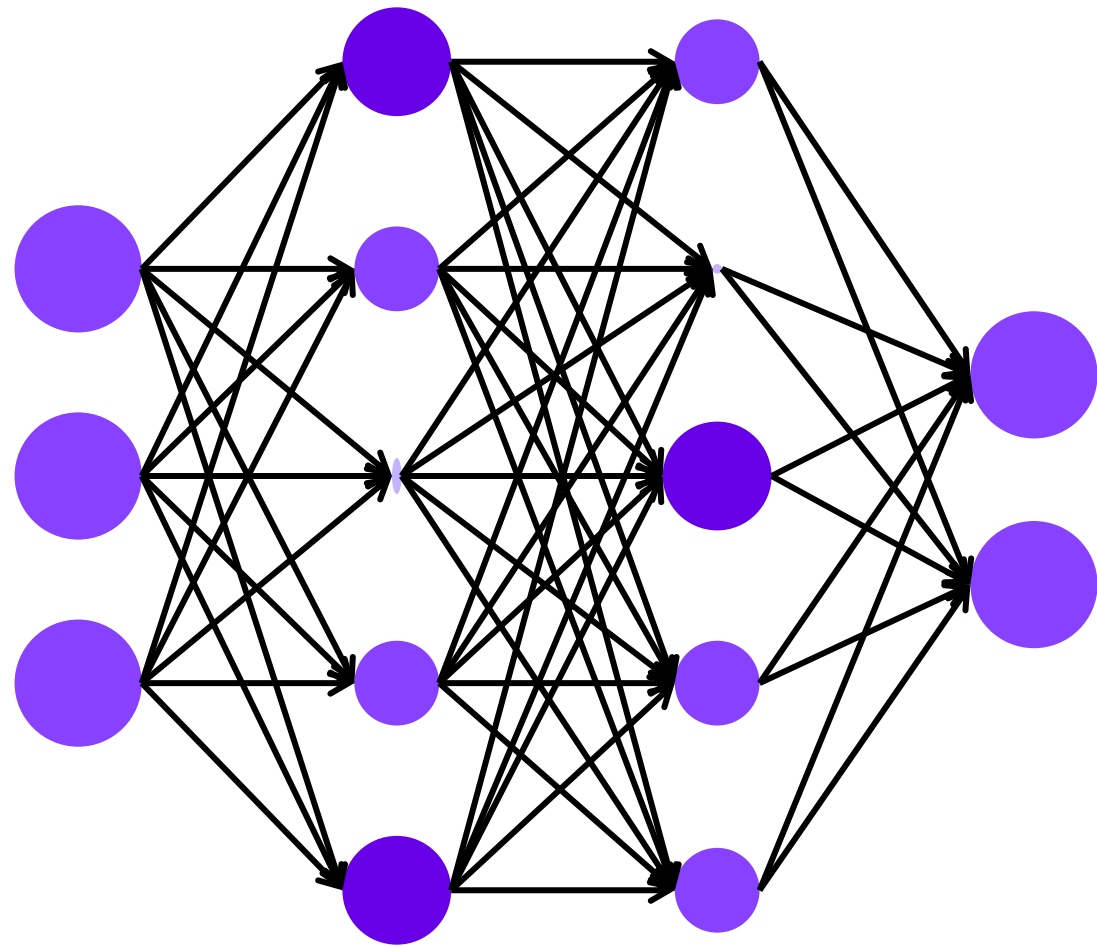


1. 가중치 정규화 (Regularization)



- 모델의 가중치 값이 서로 큰 차이를 보이는 경우, 이는 일반적으로 모델이 과도하게 복잡한 상태임을 나타낸다.
- 모델이 과도하게 복잡한 상태일 경우, **학습 데이터를 지나치게 학습** 하여 학습 데이터에만 존재하는 세부적인 패턴까지 학습한 상태일 가능성이 높다.
- **가중치 정규화 기법을 활용하여** 모델의 과도한 복잡성을 억제함으로써 과적합 현상을 감소시킬 수 있다.

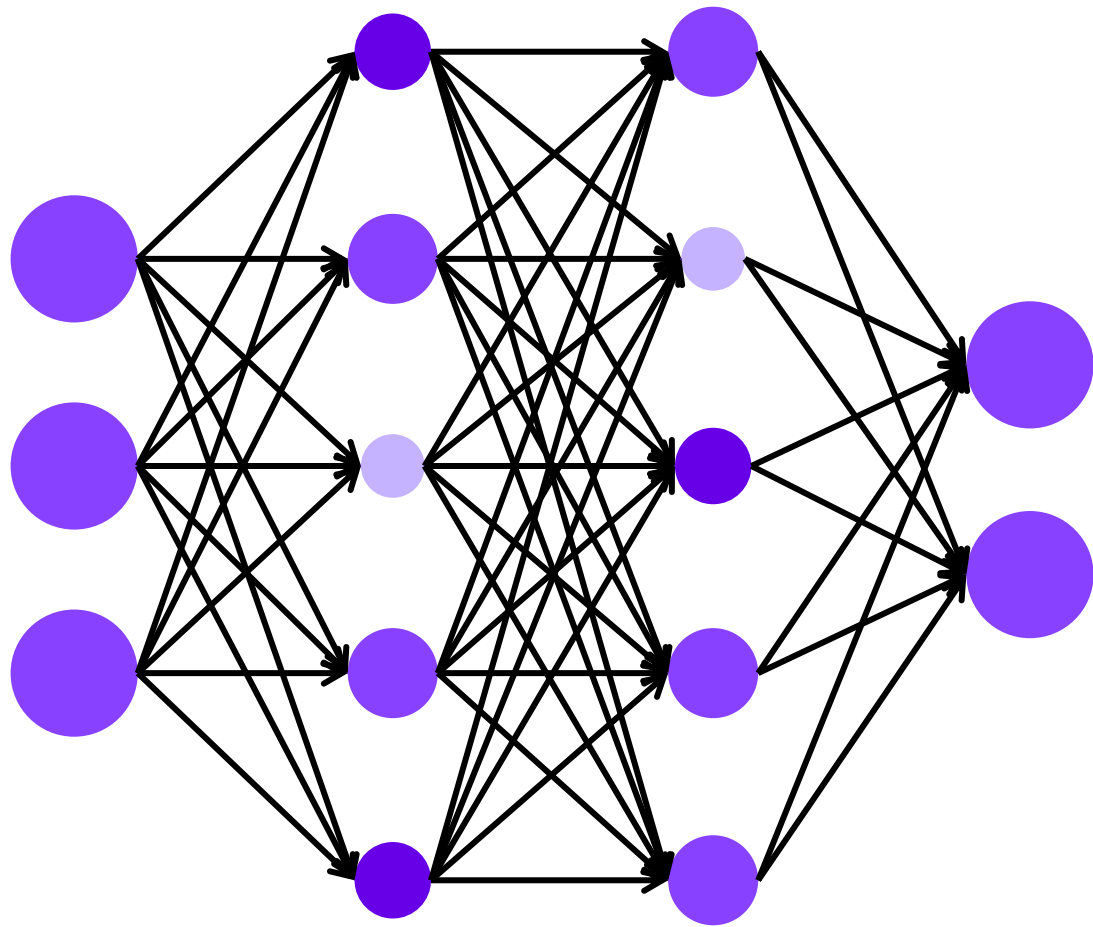
L1 정규화 (Lasso Regularization)



- 가중치의 절대값의 합의 일정 비율을 손실 함수에 더하는 정규화 기법
- 작은 가중치들이 0으로 수렴하고, 몇 개의 중요한 가중치들만 남음
- 머신러닝 에서의 L1 정규화와 본질적으로 같은 기법

L2 정규화 (Ridge Regularization)

- 가중치의 제곱의 합의 일정 비율을 손실 함수에 더하는 정규화 기법

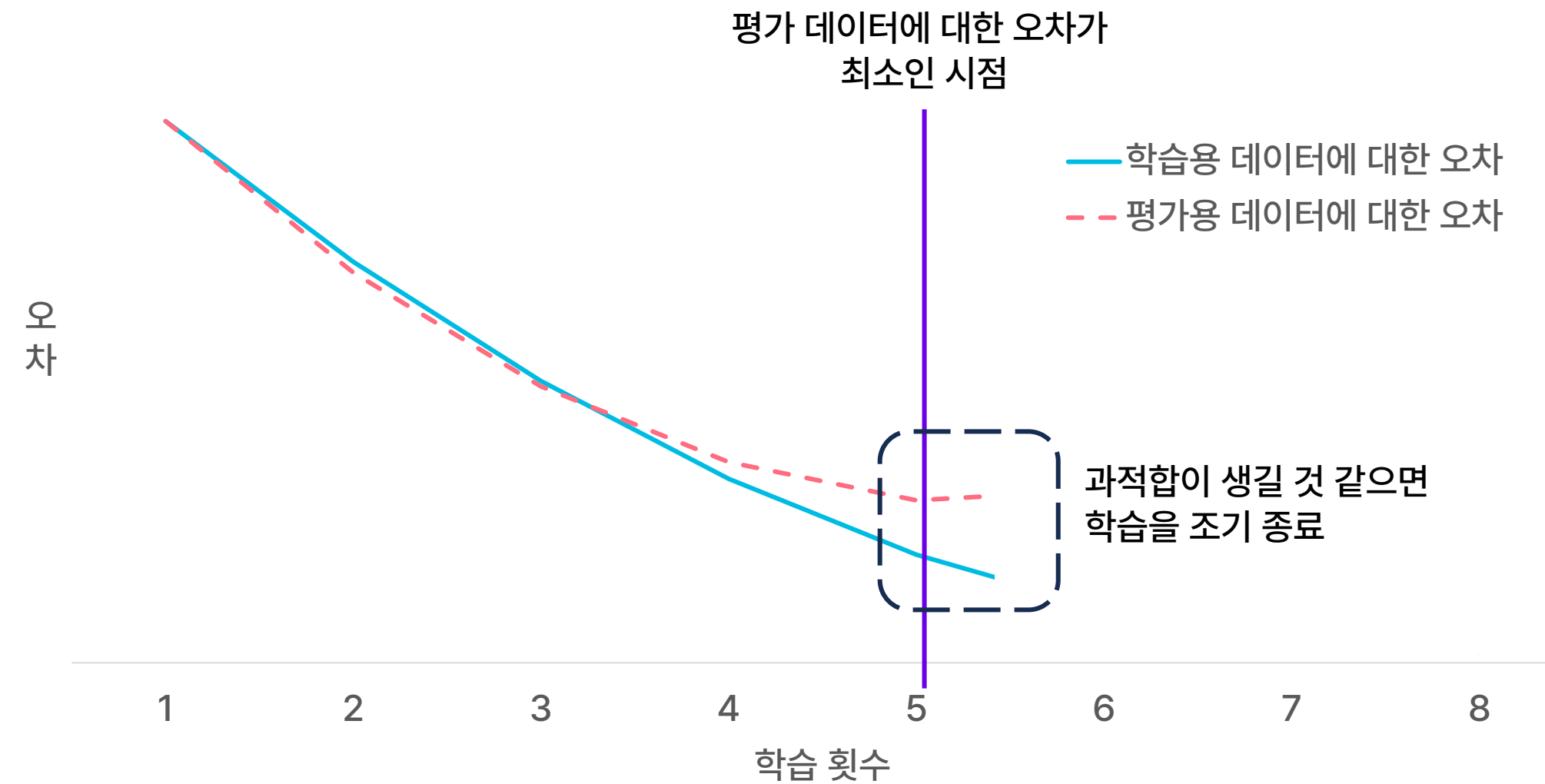


- L1 정규화에 비해 0으로 수렴하는 가중치가 적고, 큰 가중치를 더 강하게 규제

- 머신러닝 에서의 L2 정규화와 본질적으로 같은 기법



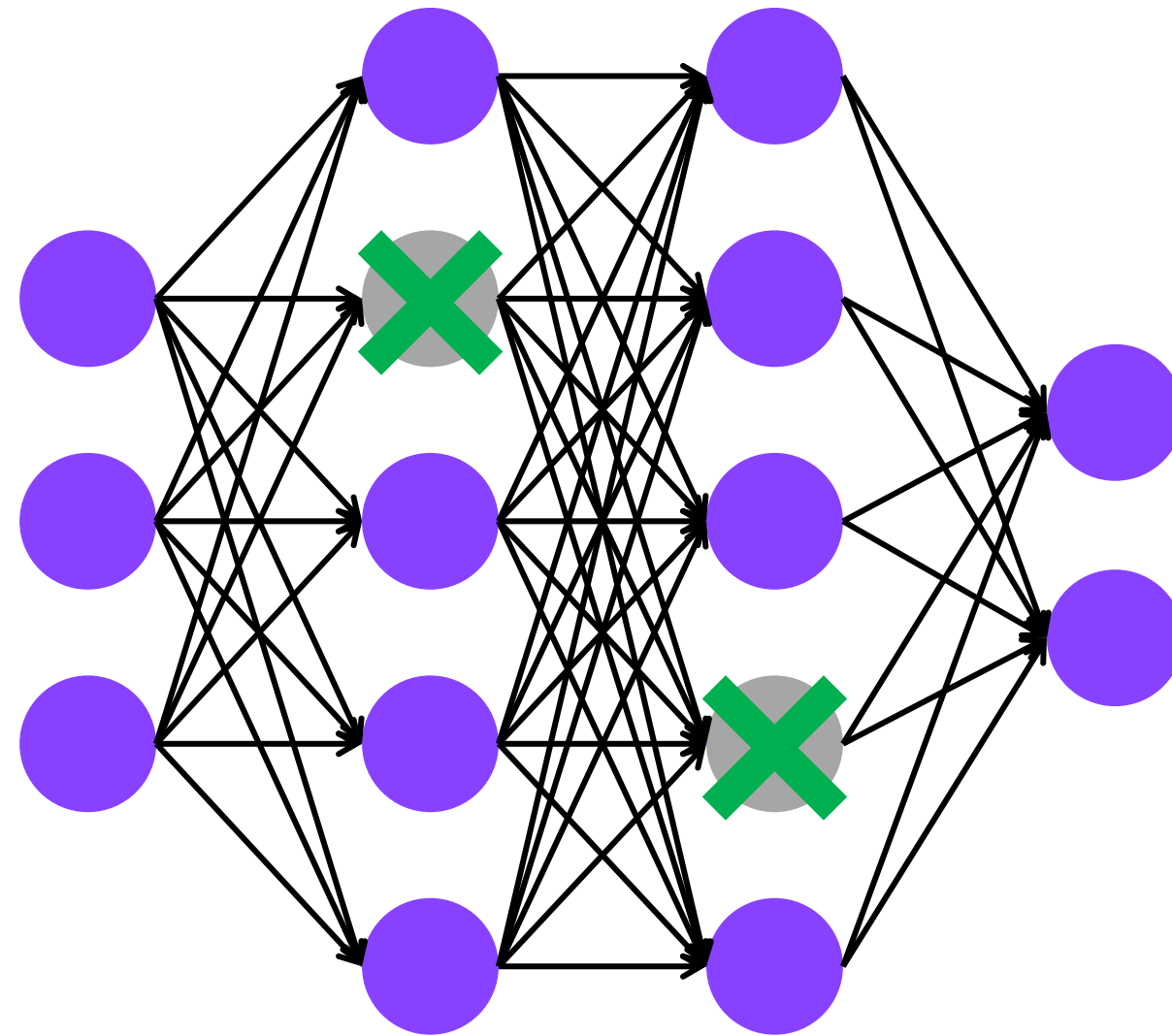
2. 조기 종료 (Early Stopping)



일정 주기마다 **검증**을 거쳐, 검증 과적합 현상이 발생하기 시작한 경우 학습을 조기 종료하는 기법



3. Dropout



학습 중 일부 퍼셉트론을 랜덤하게 비활성화 하여 모든 퍼셉트론이 과적합 되는 것을 억제하는 기법

검증 및 테스트 과정에서는 적용하지 않음